

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	M.Tech Computer Science & Engineering	Name	Pandiarajan D
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	Sem V

Experiment 2: Email Spam or Ham Classification using Naive Bayes, KNN, and SVM

Aim :-

To classify emails as spam or ham using Naive Bayes, KNN, and SVM algorithms and compare their performance.

Libraries used :-

1. Numpy
2. Pandas
3. Matplotlib
4. Seaborn
5. Sklearn

Mathematical/theoretical description of the algorithm/objective performed

The objective is to build a classification model that can accurately distinguish between spam (unwanted) and ham (legitimate) emails. We employ three different algorithms:

- **Naive Bayes:** A probabilistic classifier based on Bayes' theorem with strong independence assumptions between features. The fundamental equation is:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)} \quad (1)$$

where y is the class label and X is the feature vector. We test three variants:

- *GaussianNB*: Assumes features follow a normal distribution:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right) \quad (2)$$

- *MultinomialNB*: Used for multinomially distributed data, ideal for discrete counts (e.g., word counts):

$$P(x_i|y) = \frac{N_{x_iy} + \alpha}{N_y + \alpha n} \quad (3)$$

where N_{x_iy} is count of feature x_i in class y , N_y is total count of all features in class y , α is smoothing parameter, and n is number of features.

- *BernoulliNB*: Designed for binary/boolean features (e.g., word presence/absence):

$$P(x_i|y) = P(i|y)x_i + (1 - P(i|y))(1 - x_i) \quad (4)$$

- **K-Nearest Neighbors (KNN)**: A non-parametric, instance-based learning algorithm. It classifies a sample based on the majority class among its k nearest neighbors in the feature space. The distance between instances is typically calculated using:

- Euclidean distance (most common):

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (5)$$

- Manhattan distance:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (6)$$

We evaluate different values of k (1, 3, 5, 7) and different tree structures (KDTree, Ball-Tree) for efficiency.

- **Support Vector Machine (SVM)**: A discriminative classifier that finds an optimal hyperplane to separate classes in a high-dimensional space. The optimization problem is:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (7)$$

subject to $y_i(w \cdot x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$ for all i , where C is the regularization parameter. We test different kernels:

- *Linear*: $K(x_i, x_j) = x_i \cdot x_j$
- *Polynomial*: $K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$
- *RBF (Radial Basis Function)*: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- *Sigmoid*: $K(x_i, x_j) = \tanh(\gamma x_i \cdot x_j + r)$

Performance is evaluated using standard metrics: Accuracy, Precision, Recall, and F1-Score, along with training time where applicable.

Code Implementation

```
# Placeholder for data loading and preprocessing code
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv('spam_ham_dataset.csv')
X = data['text']
y = data['label']

# Convert text to features using TF-IDF
vectorizer = TfidfVectorizer(max_features=5000)
X_vectorized = vectorizer.fit_transform(X)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y,
                                                    test_size=0.2,
                                                    random_state=42)

# Placeholder for Naive Bayes implementation
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score, classification_report

gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()

# Fit models (converting sparse matrix to dense for GaussianNB)
gnb.fit(X_train.toarray(), y_train)
mnb.fit(X_train, y_train)
bnb.fit(X_train, y_train)

# Make predictions and evaluate
# code for KNN and SVM

# Placeholder for ROC curve and confusion matrix generation
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay, ConfusionMatrixDisplay

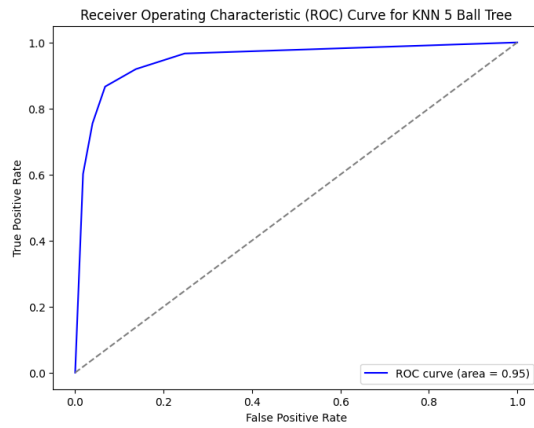
# Example for one model
RocCurveDisplay.from_estimator(mnb, X_test, y_test)
plt.title('ROC Curve - Multinomial Naive Bayes')
plt.show()

ConfusionMatrixDisplay.from_estimator(mnb, X_test, y_test)
plt.title('Confusion Matrix - Multinomial Naive Bayes')
plt.show()
```

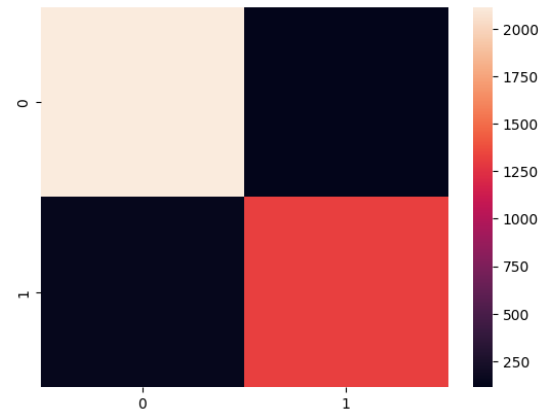
Plots and Results

KNN Model

Ball Tree with k=5



(a) ROC Curve

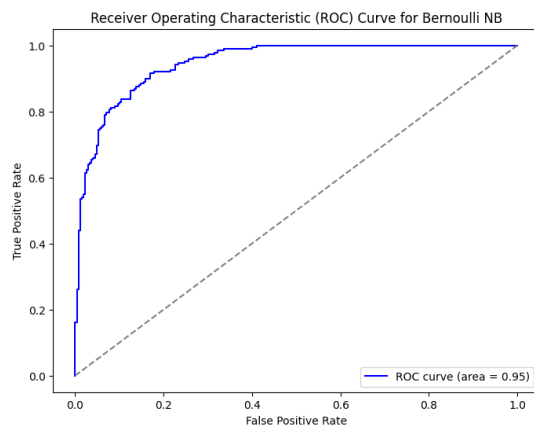


(b) Confusion Matrix

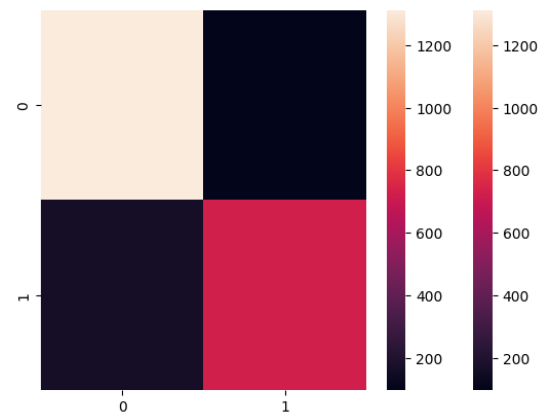
Figure 1: Evaluation metrics for Ball Tree model with k=5

Naive Bayes Model

Bernoulli Naive Bayes



(a) ROC Curve

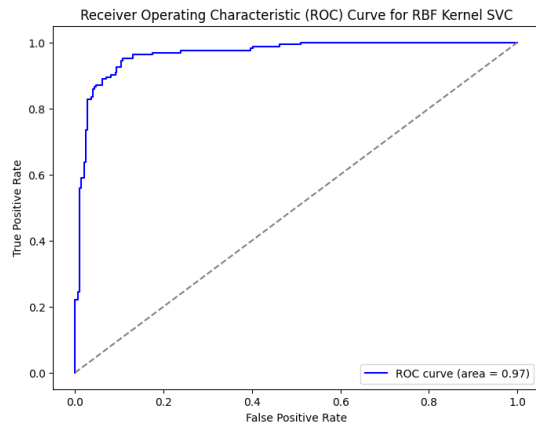


(b) Confusion Matrix

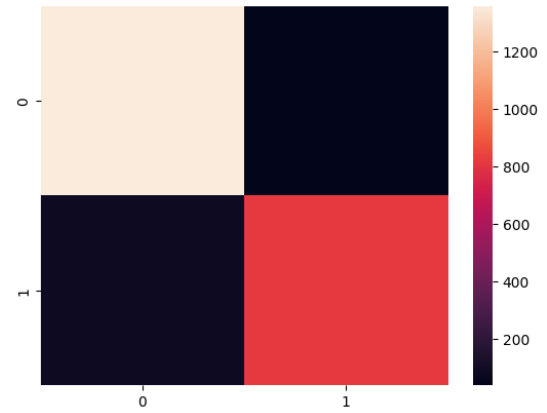
Figure 2: Evaluation metrics for Bernoulli Naive Bayes model

Support Vector Classifier

SVC with RBF Kernel



(a) ROC Curve



(b) Confusion Matrix

Figure 3: Evaluation metrics for SVC with RBF kernel

Performance Comparison of Naïve Bayes Variants

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.82	0.87	0.87
Precision	0.71	0.92	0.87
Recall	0.96	0.76	0.80
F1 Score	0.82	0.83	0.83

KNN Performance for Different k Values

k	Accuracy	Precision	F1 Score	Recall
1	0.90	0.86	0.88	0.89
3	0.88	0.85	0.86	0.86
5	0.88	0.87	0.86	0.85
7	0.88	0.86	0.85	0.84

KNN Comparison: KDTree vs BallTree at $k = 1$

Metric	KDTree	BallTree
Accuracy	0.90	0.90
Precision	0.86	0.86
Recall	0.89	0.89
F1 Score	0.88	0.88
Training Time (s)	0.0157	0.0108

KNN Comparison: KDTree vs BallTree at $k = 3$

Metric	KDTree	BallTree
Accuracy	0.88	0.88
Precision	0.85	0.85
Recall	0.86	0.86
F1 Score	0.86	0.86
Training Time (s)	0.0263	0.0249

KNN Comparison: KDTree vs BallTree at $k = 5$

Metric	KDTree	BallTree
Accuracy	0.88	0.89
Precision	0.87	0.87
Recall	0.85	0.85
F1 Score	0.86	0.86
Training Time (s)	0.0212	0.0249

KNN Comparison: KDTree vs BallTree $k = 7$

Metric	KDTree	BallTree
Accuracy	0.88	0.88
Precision	0.86	0.86
Recall	0.84	0.84
F1 Score	0.85	0.85
Training Time (s)	0.0245	0.0286

SVM Performance with Different Kernels and Parameters

Kernel	Hyperparameters	Accuracy	F1 Score	Training Time (s)
Linear	$C = 10$	0.91	0.88	1.245
Polynomial	$C=10$, degree=2, gamma='scale'	0.90	0.85	2.112
RBF	$C=10$, gamma=1.0	0.92	0.88	1.876
Sigmoid	$C=10$, gamma=1.0	0.88	0.83	1.543

Cross-Validation Scores for Each Model (K=5)

Fold	NB Accuracy(Bernoulli)	NB Accuracy(Gaussian)	NB Accuracy(Multinomial)
Fold 1	0.8806	0.8219	0.8719
Fold 2	0.8935	0.8065	0.8935
Fold 3	0.8891	0.7946	0.8891
Fold 4	0.8870	0.8228	0.8913
Fold 5	0.8902	0.8337	0.8859
Average	0.8881	0.8159	0.8863

Cross-Validation Scores for Each Model (K=5)

Fold	KNN Accuracy (k=1)	SVM Accuracy (RBF)
Fold 1	0.8849	0.9349
Fold 2	0.8967	0.9348
Fold 3	0.8967	0.9337
Fold 4	0.9043	0.9337
Fold 5	0.9033	0.9348
Average	0.8972	0.9344

Observations

- **Overall Performance:** The SVM with an RBF kernel achieved the highest accuracy (0.92) and consistent performance across cross-validation folds (avg. 0.934), making it the best-performing model for this dataset.
- **Naive Bayes:** The Multinomial and Bernoulli variants performed similarly and significantly better than Gaussian NB. This suggests the text data is better modeled by discrete distributions (word counts or presence) rather than a continuous Gaussian. Multinomial NB had very high precision (0.92) but lower recall, meaning it was excellent at not labeling ham as spam but missed more actual spam.
- **KNN:** Performance was very good and stable across different k values, with k=1 being slightly superior. The choice of tree algorithm (KDTree vs. BallTree) had a negligible impact on predictive performance but a small, variable effect on training time.
- **SVM:** The RBF and Linear kernels yielded the best results, significantly outperforming the Polynomial and Sigmoid kernels. This indicates that the data is either linearly separable or requires a smooth non-linear boundary for optimal separation.
- **Cross-Validation:** The 5-fold cross-validation results confirm the stability of the models. SVM (RBF) and KNN (k=1) show the highest and most consistent scores, indicating they generalize very well to unseen data.

Learning-Outcomes

- I learned the practical implementation and comparative analysis of Naive Bayes variants (Gaussian, Multinomial, Bernoulli) for text classification.
- I understood the effect of the hyperparameter 'k' on the KNN algorithm's performance and the trade-off between model complexity and generalization.
- I gained experience in implementing SVM with different kernels and tuning hyperparameters like C and gamma to optimize performance.
- I practiced using cross-validation to robustly evaluate model performance and ensure results are not dependent on a single train-test split.
- I learned the importance of choosing the right algorithm based on the data characteristics (e.g., using Multinomial/Bernoulli NB for text data instead of Gaussian NB).